Plugin Architectures with Rust

an Analysis of Obstacles and Prospects

20.05.2026

Contents

1 Introduction

Plugin Systems

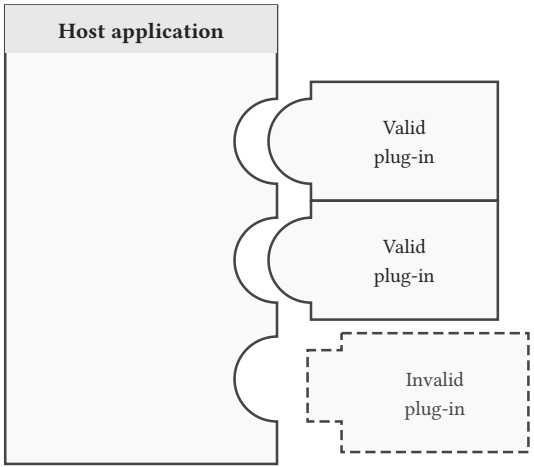


Figure 1: Computer Software that adds new functionality to an existing application without altering the host program itself

Criteria

Performance Overhead

Resulting from the required overhead, how slow is the execution of the plugin compared to the rust (static) native solution.

This is measured by using a average data fusion with 1 and with 1.000.000 data points and expressed as **percentages**. Quantified by using the criterion benchmark.

Development Complexity

- Learning curve of the implementation
- Quality of the Documentation and examples
- Required amount of work and consequences for setup, configurations and integration

Criteria (ii)

(Language) Limitations

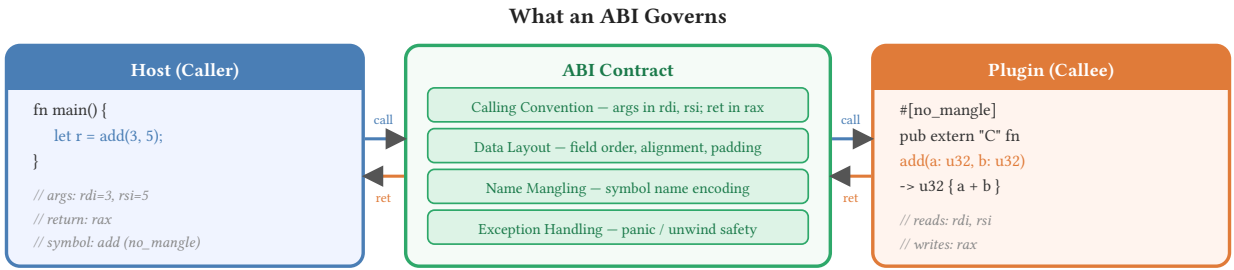
How much does this approach limit the usage of the rust languages features and ecosystem.

Interoperability

How well can this approach work with other libraries, tools and especially languages.

2 Application Binary Interface (ABI)

Application Binary Interface



Contract specification: Calling conventions, Data Layout, Name Mangling, Exception Handling, etc.

Dynamically loading plugin into memory follows:

- **Discovery**: host locates plugin binary file
- **Loading**: host loads plugin into memory
- **Resolution**: host resolves symbol address
- **Execution**: host invokes plugin function

(Unstable) Rust ABI

2.b.a Demonstration of dynamic linking and loading

```rust
// In base library (used by application and plugin):
trait Fuser {
  fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>; }
```

```rust
// In plugin:
struct AveragePlugin;
impl Fuser for AveragePlugin { ... }

#[unsafe(no_mangle)] // ensure name can be found in binary
pub fn average_plugin() -> Box<dyn Fuser> { Box::new(AveragePlugin::new()) }
```

(Unstable) Rust ABI (ii)

Rust

```rust
// In application:
type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>; // use Rust ABI
let plugin: Plugin<FuseFunc> = Plugin::new( // internally uses `libloading`
  &plugin_path.into(),
  "average_plugin".as_bytes()
).unwrap(); // Discovery, Loading and Resolution

let fuser: Box<dyn Fuser> = plugin.get();
let sd = [ SensorData::new(24.0, 0.01), ... ];
let res = fuser.fuse(&sd); // Execution
```

(Unstable) Rust ABI (iii)

2.b.b .... but it's Unstable

The implementation itself is not complex, but...

- Compiler version changes **may** impact the generated binary
- Changes to structs, traits and enums **may** impact the generated binary
- Base library, application and plugins need to run on the **same compiler version**

But we can use the **C ABI** (next slides)! But representing Rust in C (and vice versa) is very difficult and will introduce trade-offs. Some of them are:

- How are structs, traits and enums represented?
- What happens to VTables?
- What about Lifetimes and Ownership?
- ...

abi_stable crate

- Usage of `#[sabi_trait]`, `#[export_root_module]`, `#[sabi_extern_fn]`, ... attribute macros for creating FFI-safe (Foreign Function Interface) trait objects
- Allows for a lot of customization when specifying the FFI
- Provides ABI-stable alternatives to many *std-types*, but also some *external types*
- Widely used crate with a lot of community support

but:

- Many hidden structs and types are generated and are required for further use, therefore it is overwhelming at first
- Usage of `async` is not possible, because a FFI-safe wrapper for `Future` is not supported.
- Github repository seems to be dormant.

stabby crate

- Easy to get started as primarily `#[stabby::stabby]`, `#[stabby::export]` and `stabby::dynptr!(...)` are used.
- Good explanation of the concepts behind ABI development (and why Rust doesn't like the C ABI and vice versa)
- Provides ABI-stable alternatives to common types
- Built for performance
- "Compiler-change-proof ABI-stability is proven statically through the type system."

but:

- relatively new and therefore not a lot of community engagement (yet)
- had some breaking changes between versions due to a rust compiler update
- documentation can be spotty at times

Comparison ABI crates

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	*9%*	0	0	+

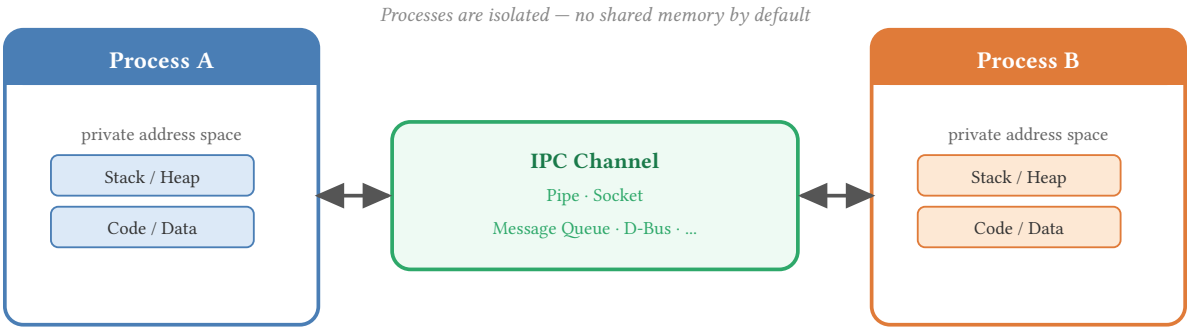
Table 1: Summary of ABI results.

Crate	Macro Calls	Type Changes
unstable_abi	1	0
abi_stable	*16*	2
stabby	3	2

Table 2: Code complexity depicted as amount of line or type changes.

3 Interprocess Communication (IPC)

Interprocess Communication (IPC)



Biggest takeaway: Shared memory is not accessible by default for this approach. Therefore (de-) **serialization** is required.

rustbridge crate

High-level JSON, native Rust speed — Work with serde types, not raw pointers!

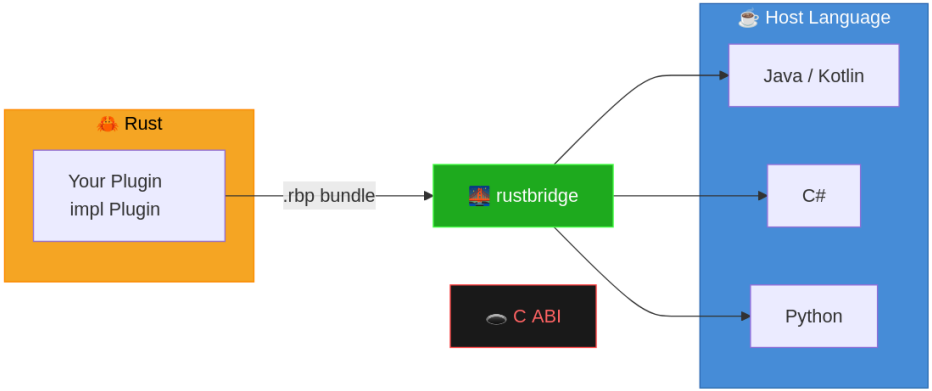


Figure 4: A rustbridge plugin can be bundled into a .rbp file and used across different languages.

rustbridge crate (ii)

- Stable C ABI — Plugins work regardless of your Rust compiler version or optimization flags
- Managed lifecycle: Startup, shutdown, including logging callbacks
- Communication with Request/Response Headers (JSON or Binary)
- Usage of CLI tools to create the `.rbp` bundle
- Excellent technical documentation and examples

but:

- No shared memory, therefore performance loss due to serialization
- Binary approach requires manual conversion of bytes to types
- Requires strict separation of plugin and application

Comparison rustbridge JSON vs Binary

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
rustbridge (JSON)	*537%*	+	0	++
rustbridge (Binary)	25%	-	0	++

Table 3: Summary of IPC results.

4 Conclusion

Conclusion

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	*25%*	0	0	+
stabby	9%	0	0	+
rustbridge (JSON)	537%	+	0	++
rustbridge (Binary)	*25%*	-	0	++

Table 4: Summary of results.

- Development complexity and Limitation aren't fine grained enough
- Updating the plugin and host hasn't been tested

Conclusion (ii)

From the 2025 Rust Survey:

- 14% of respondents stated that a stable ABI would unblock their use cases - around 24% state that it would improve their code
- 13% consider implementing a dynamic library plugins a significant problem

Currently no visible signs that a stable ABI is coming anytime soon.

Thank you for your attention!
Any questions?